ANKARA YILDIRIM BEYAZIT UNIVERSITY FACULTY OF AERONAUTICS AND ASTRONAUTICS

AEE323 ENGINEERING EXPERIMENTATION AND DATA ANALYSIS

GROUP 7 PROJECT REPORT

SEYYİTHAN ARAZ 21110111044
BUSE IRMAK 21110111033
İBRAHİM HAKKI KOÇ 22110111023
MİKAİL MİRAÇ BOZDOĞAN 20110111015

Dec 12, 2024

Contents

1 Introduction **3**

2 Methodology **3**
2.1 Data and Frequency Analysis 4
2.2 Sampling Frequency 4
2.3 Low-Pass Filter Design 4
2.4 Defining Filter Parameters 4
2.4.1 Cutoff Frequency 5
2.4.2 Transition Width 7
2.4.3 Order 9
2.4.4 Filter Kernel 10
2.5 Applying Filter To Data 12
2.6 Applying Inverse Fourier Transform 12

3 Results and Discussion **12**
3.1 Filtering Process and Results 12
3.2 Error Analysis 14
3.3 Inverse Fourier Transform and Validation 15
3.4 Noise Analysis and Potential Causes 17
3.5 Thinking about the Cause and Significance of Noise 17

A MATLAB Code **18**

Nomenclature

Symbol	Description
f_s	Sampling frequency (Hz)
Δt	Time step between samples (sec)
$BIBO$	Bounded input-bounded output
f_{cutoff}	Cutoff frequency (Hz)
$transw$	Transition width for filter design
$RMSE$	Root Mean Square Error
FFT	Fast Fourier Transform
$IFFT$	Inverse Fast Fourier Transform
y	Measured signal (with noise)
y_{Orig}	Original signal (without noise)
y_{Filt}	Filtered signal (low-pass filtered)
$invFilt$	Inverse filtered signal
N	Number of samples
SG	Savitzky-Golay filter
EEG	Electroencephalography
ECG	Electrocardiography
NIRS	Near-Infrared Spectroscopy
fMRI	Functional Magnetic Resonance Imaging

1 Introduction

Filter is a very important component in the signal processing field. In broader sense, a filter is a device that discriminates, according to some attribute of the object at its input, what passes through it [1]. In the project, we applied a filter application to reduce the noise in the data set given to us. While doing this, we filtered the noise with a low pass filter (LPF). Low pass filters are used in signals where high frequency elements need to be filtered. The low pass filter passes low-frequency signals but attenuates signals with frequencies higher than the cut-off frequency. Examples of these are filtering applications in electrical circuits. In such applications, system data are measured and digitised. The desired filter type is applied and some changes are made. In our project, we uploaded the data given to us as 'GroupData7' file to MATLAB. The file consists of time (in seconds), measurement data (with noise), original data (without noise) . Time-domain representation of these data (original and measured) are shown in Figure 1.

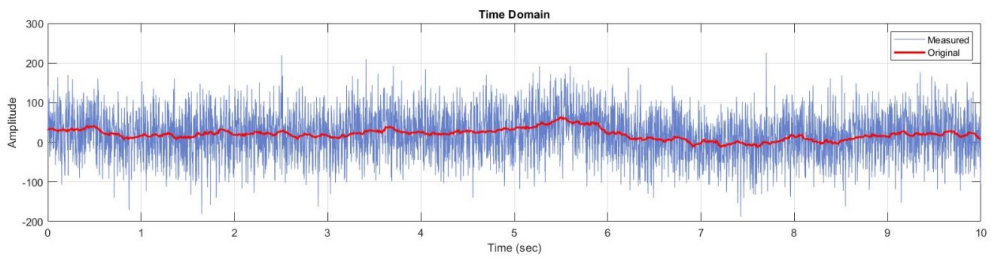

Figure 1: Measured and Original Signal (Time Domain)

Also, we can see power spectrum of measured signal as shown in Figure 2.

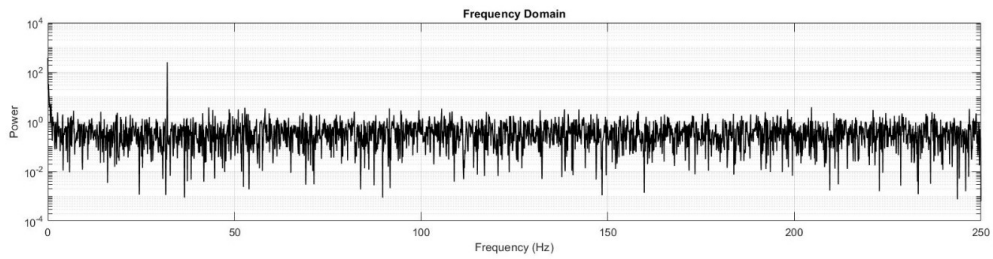

Figure 2: Power Spectrum of Measured Signal

2 Methodology

We imported the data in the 'GroupData7' file given to us into MATLAB. The .*mat file consists of time (in seconds), measured(with noise), and original (without noise) data, as mentioned. Of these, the measured values are the data with noise, while the original values are the data without noise. The main problem we are studying in this project is to optimally reduce the effects caused by noise to the measured signal. The steps we follow for this are as follows.

2.1 Data and Frequency Analysis

Firstly, the basic characteristics and distributions of the measured and original signals were discussed. In this step, we decided what the causes of the noise could be and how we could apply our filtering method in this system. Then, we analyzed the frequency and determined the points where we could make cutoffs. We applied the inverse Fourier transform.

2.2 Sampling Frequency

Before designing the filter,we determined sampling frequency with the help of given *timevec* data .Sample rate, also known as sampling frequency (f_s), refers to the number of samples taken per second from a continuous signal in order to create a discrete signal. The typcial method of obtaining a discrete-time representation of a continuous-time signal iis through sampling,where in a sequence of samples, $x[n]$, is obtained from a continuous-time signal $x_c(T)$ according to relation;

$$x[n] = x_c(nT), \quad -\infty < n < \infty.$$

T is sampling period,and its reciprocal, $f_s = 1/\mathrm{T}$ is the sampling frequency,in samples per second.[2]

$$fs = \frac{1}{T}$$

In MATLAB, we determine the sampling frequency f_s using the following line of code:

```
fs = 1 / mean(diff(time));
```

2.3 Low-Pass Filter Design

In this study, we implemented a low-pass filter to minimize high-frequency noise in the signal. Digital filters are systems that perform mathematical operations on sampled discrete-time signals to reduce or enhance specific aspects of those signals. They are highly programmable and accurately handle low-frequency signals. Digital filters are classified into two types: Infinite Impulse Response (IIR) and Finite Impulse Response (FIR). IIR filters have infinite-length impulse responses, while FIR filters have finite-duration responses. FIR filters are the preferred choice when an exact linear phase response is required.[3] The comparison of FIR and IIR filters, shown in Table 1, highlights key differences. FIR filters are simple to design and they are guaranteed to be bounded input-bounded output (BIBO) stable. FIR filters also have a low sensitivity to filter coefficient quantization errors.[4]

Feature	**FIR**	**IIR**
Kernel Length	Longer	Shorter
Stability	Intrinsically Stable	Dependent on Design
Phase Response	Linear	Nonlinear
Efficiency	Higher Computation	Faster Execution

Table 1: FIR vs. IIR Filter Comparison

The linear-phase characteristic of FIR filters avoids phase distortion, making them ideal for this application. For its linear phase response, stability, and ease of implementation, we chose an FIR filter.

2.4 Defining Filter Parameters

The filter was designed defining followed parameters. We experimented with different parameters, such as cutoff frequency (f_{cutoff}), transition width ($transw$), and filter order, to observe their effects on the filtered signal. Each of these parameters was tested with four different values to thoroughly analyze their impact on the filter's performance. Below, we discuss each of these parameters and show their results with various figures.

2.4.1 Cutoff Frequency

The cutoff frequency (f_{cutoff}) determines the point where the filter attenuates higher frequencies.As we can see in Figure 3, there is a distinct peak at a frequency of 32 Hz. So this means that we should choose our cutoff frequency below 32 Hz.

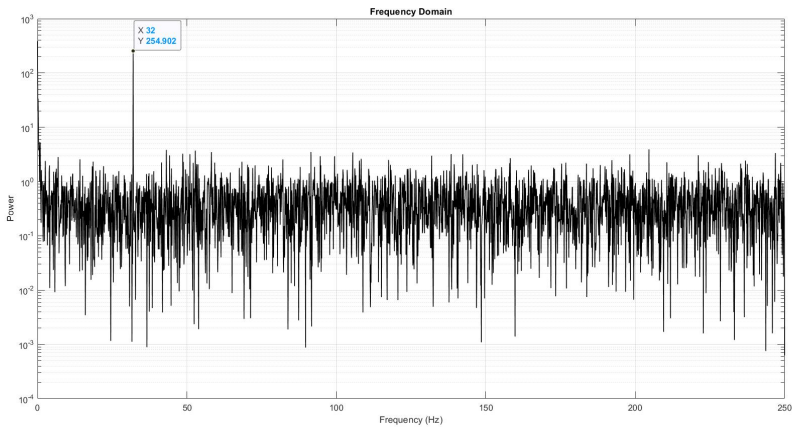

Figure 3: Power Spectrum of the Measured Signal with the Peak Indicated

When the sampling frequency is more than twice the cut-off frequency, the sampling sequence can be restored to the original waveform.[5] In this study, we tested four different cutoff frequencies: 15 Hz, 20 Hz, 25 Hz, and 30 Hz.

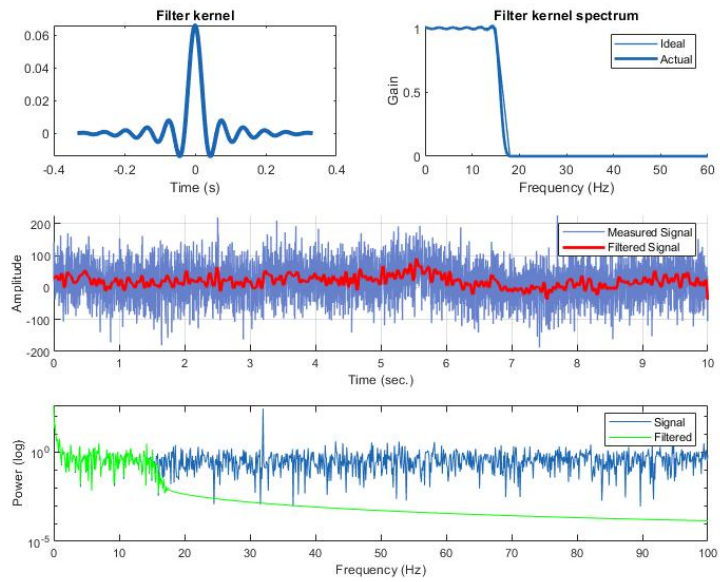

Figure 4: When Cut Off Frequency Is Equal To 15 Hz.

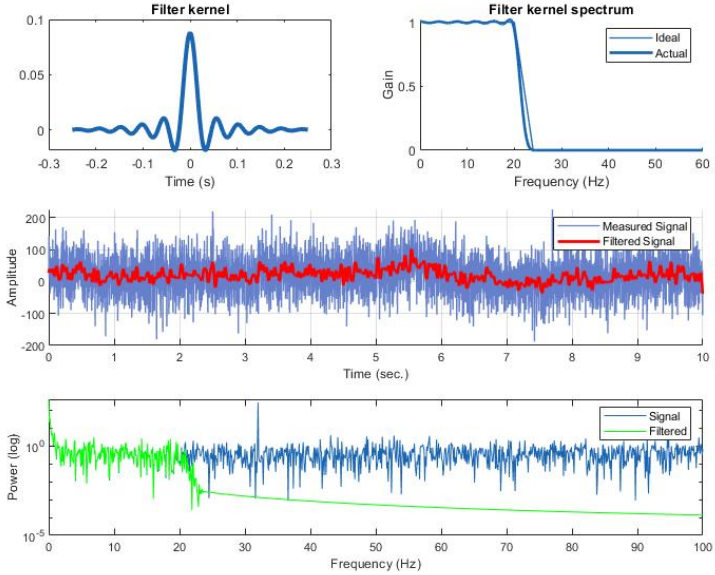

Figure 5: When Cut Off Frequency Is Equal To 20 Hz.

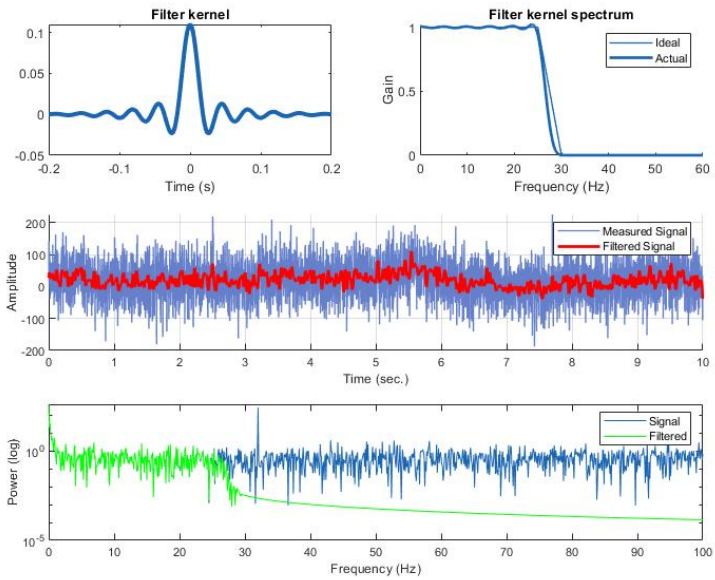

Figure 6: When Cut Off Frequency Is Equal To 25 Hz.

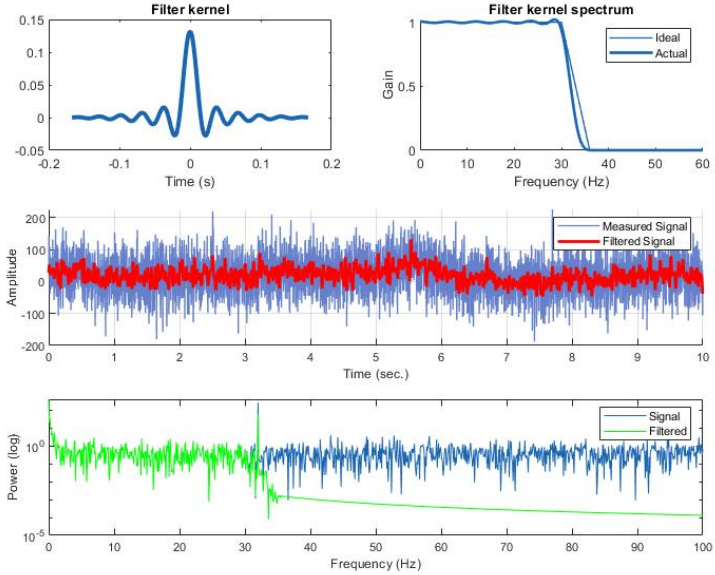

Figure 7: When Cut Off Frequency Is Equal To 30 Hz.

Figures 4 through 7 show results for the different cutoff frequencies. The original, noisy, and filtered signals are compared to demonstrate how varying the cutoff frequency affects the signal. We can see that if we choose cutoff frequency to low (Figure 4), we ignore valuable data above this frequency. On the other hand,if we choose cutoff frequency to high (Figure 7) filter does not work properly since we include unwanted data above 32 Hz.

2.4.2 Transition Width

Transition width (*transw*) is a critical parameter that determines the range where a filter transitions from the passband to the stopband. A narrower transition width results in a sharper cutoff but may introduce undesirable effects such as ringing. The impact of various transition widths (0.1, 0.2, 0.3, and 0.5) on filter performance was tested.

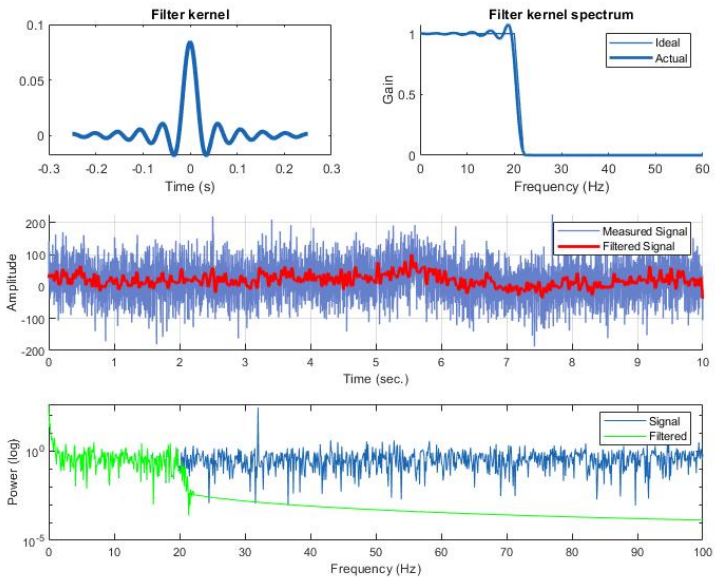

Figure 8: When Transition width is equal to 0.1 value.

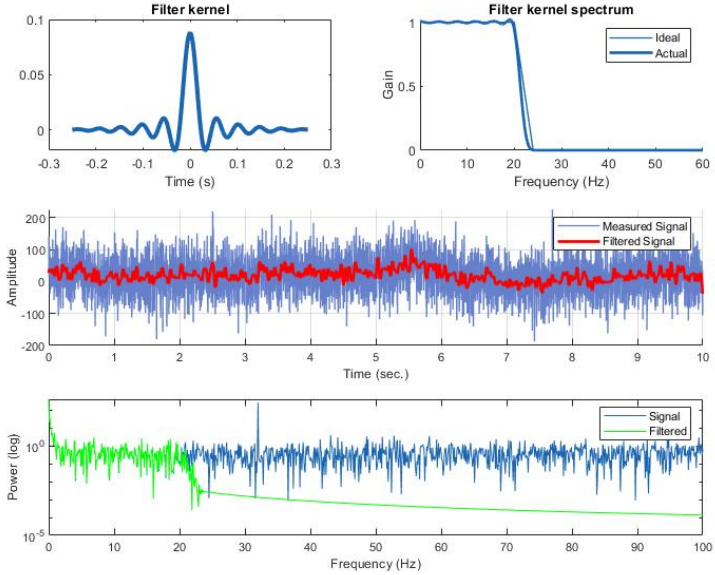

Figure 9: When Transition width is equal to 0.2 value.

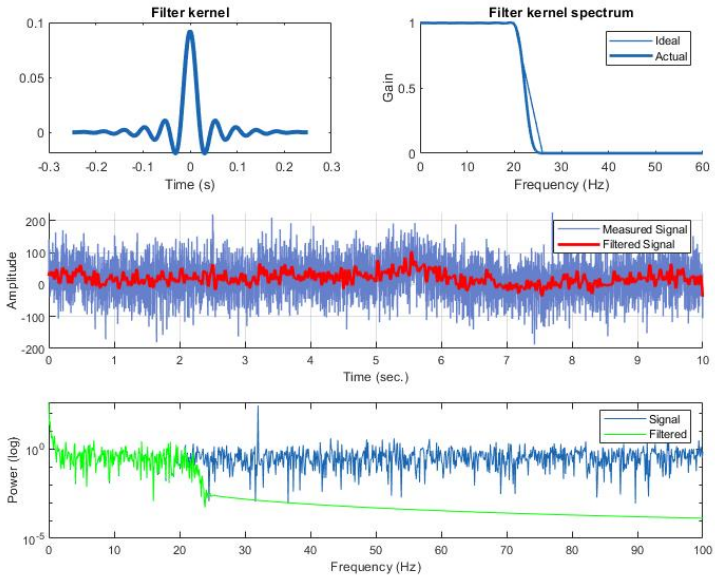

Figure 10: When Transition width is equal to 0.3 value.

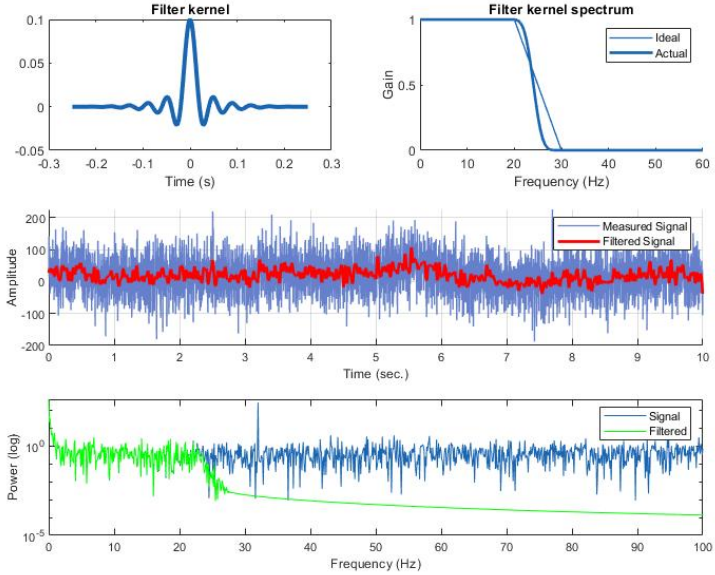

Figure 11: When Transition width is equal to 0.5 value.

Figures 8 to 11 show the response of the signal after filtering with different pass widths. The wider the pass width, the wider the range of frequencies that can be passed through the filtering process. A wider frequency range results in more frequencies being observed at the same sampling rate and a smoother graph. However, when the pass width is chosen too large, the shape of the signal can become difficult to analyze. Also, a wider frequency range requires more computational resources.

2.4.3 Order

The filter order determines the number of taps in an filter with higher orders resulting in a sharper cutoff but with increasing order the arithmetic operations go on increasing leading to complexity in calculation[8]. In this study, we tested four different filter orders to see effect of each order on filter. The filter order was defined using the MATLAB code:

```
order = round(10 * fs / fcutoff);
```

where f_s is the sampling frequency and f_{cutoff} is the cutoff frequency. The effects of each order on both time-domain and frequency-domain performance were observed and analyzed.

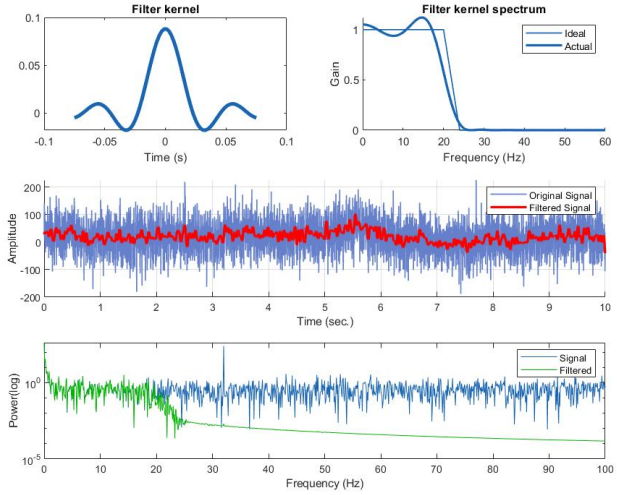

Figure 12: When Order Is Equal To 3

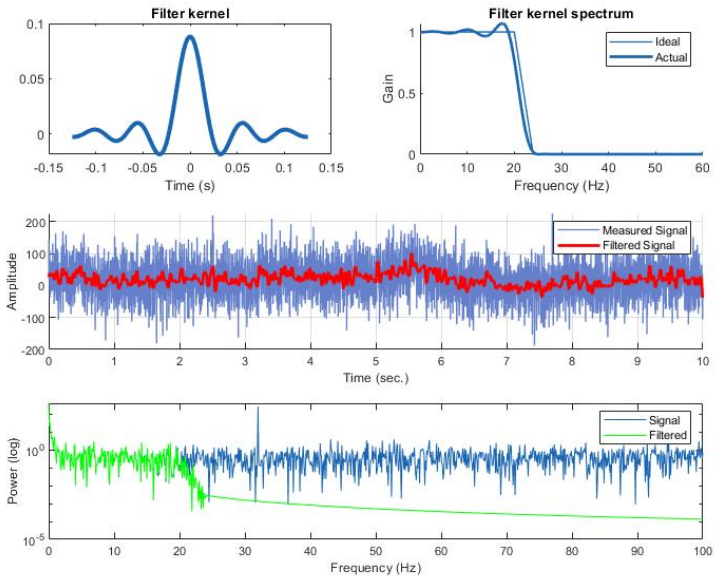

Figure 13: When Order Is Equal To 5

Figures 12 through 15 show the responses for different orders.

2.4.4 Filter Kernel

The filter kernel was generated using MATLAB's firls function [6]. This kernel represents the filter's impulse response and dictates how the input signal is processed.

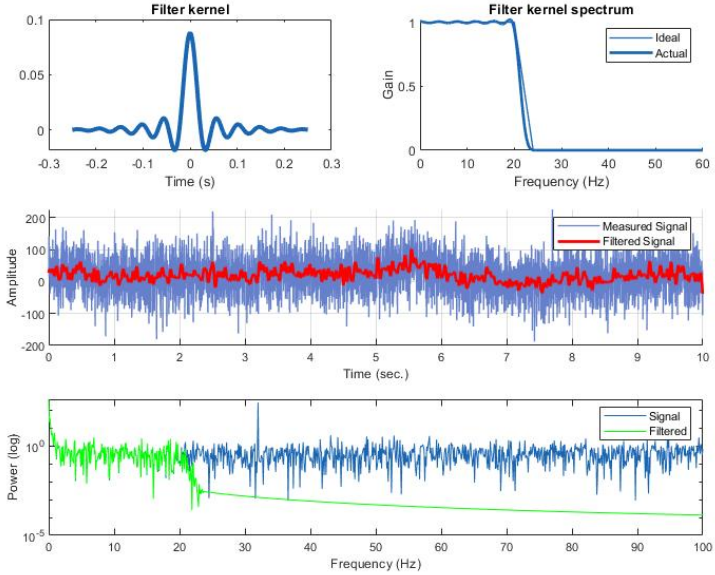

Figure 14: When Order Is Equal To 10

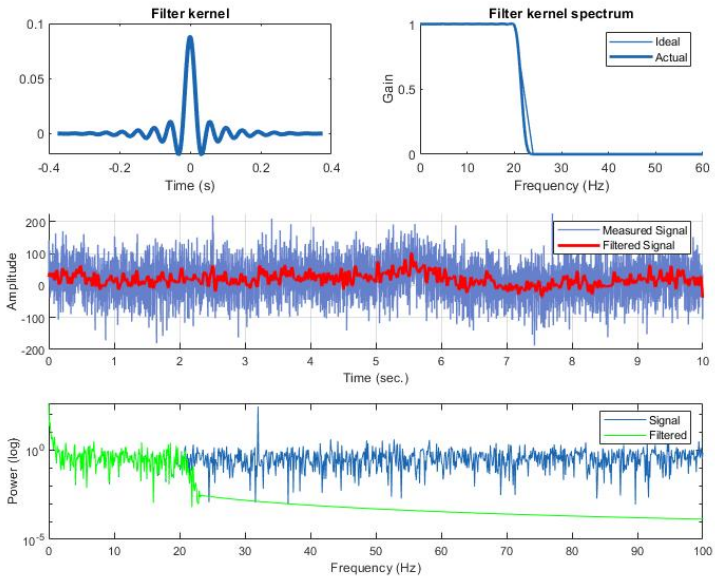

Figure 15: When Order Is Equal To 15

```
shape   = [1 1 0 0];
frex    = [0 fcutoff fcutoff+fcutoff*transw fs/2] / (fs/2);

% Filter kernel
filtkern = firls(order, frex, shape);

% Filter kernel power spectrum
filtkernX = abs(fft(filtkern, npnts)).^2;
```

Figure 16: MATLAB code for generating filter kernel and its power spectrum.

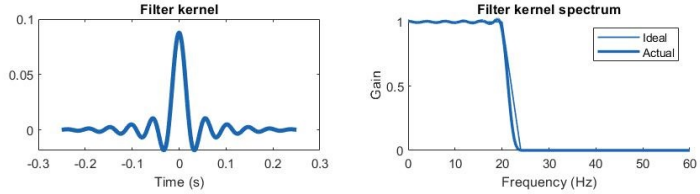

Figure 17: Time-domain representation of the filter kernel and its power spectrum.

2.5 Applying Filter To Data

After designing our filter we apply the filter to the measured signal data.The MATLAB code used for this operation is below.

```
% Applying filter to the data
yFilt = filtfilt(filtkern, 1, y);
```

Figure 18: MATLAB code for applying filter to data

The kernel was applied to the noisy signal using MATLAB's "filtfilt" function. This function applies a zero-phase filtering process by filtering the signal forward and backward.

2.6 Applying Inverse Fourier Transform

In this step, the inverse Fourier transform (IFFT) is applied to the filtered signal to convert it back into the time domain. This process shows that the filtering doesn't change the original signal's characteristics while removing noise. The MATLAB code used for this operation is shown below.

```
% Apply inverse Fourier transform to the filtered data
invFilt = ifft(fft(yFilt));
```

Figure 19: MATLAB code for applying inverse Fourier transform

3 Results and Discussion

After applying the methodology described earlier, we obtained the following results for the filtering process and its impact on the measured signal. Below, we also discuss key aspects of the methodology and results to provide a comprehensive understanding.

3.1 Filtering Process and Results

We designed a low-pass FIR filter using MATLAB for signal filtering. The cutoff frequency was set to 20 Hz, the transition width to 0.2, and the filter order was chosen to be 10. After applying these parameters, our results were as shown in Figure 20.

We choose cutoff frequency to be 20 Hz based on our observation of the results with different parameter values. This frequency was determined to be the most optimal, as it effectively filtered out the peak at 32 Hz and higher frequencies, while preserving the important data before this point. .When we repeated the observations by varying the transition width, we see that most ideal choose is 0.2.Then we choose order value to be 10.

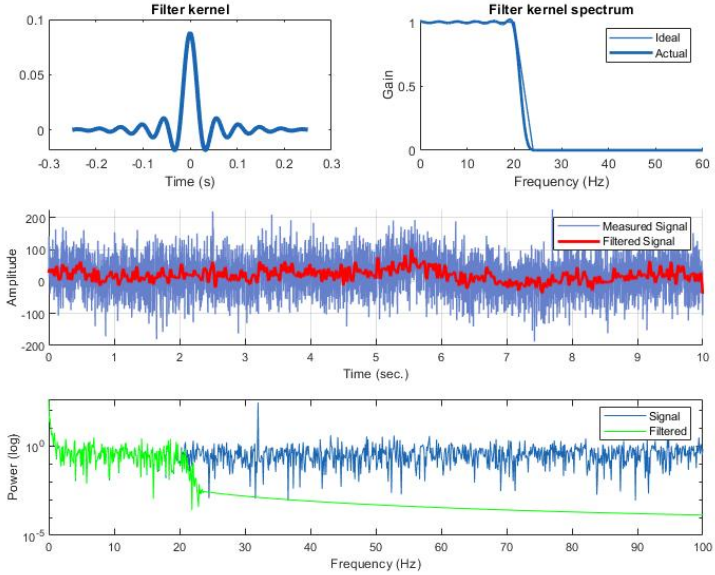

Figure 20: Filter kernel and its power spectrum above comprasion of filtered and measured signal in time and frequency domain results.

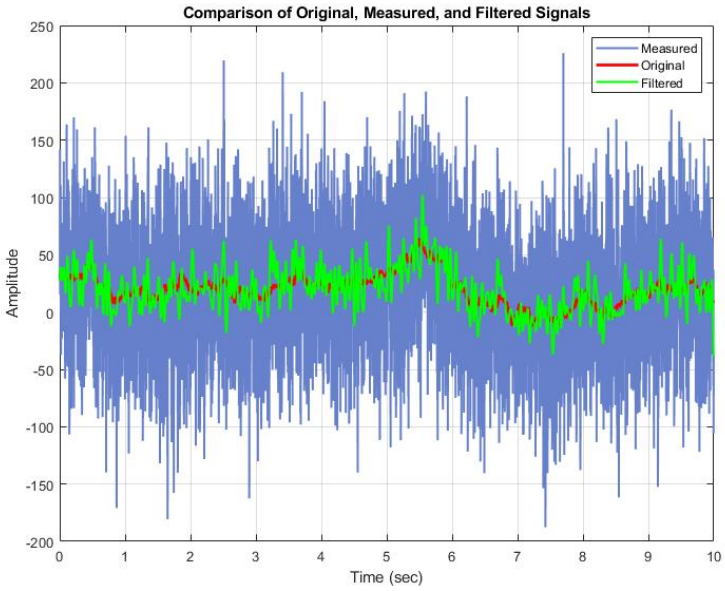

Figure 21: Time domain comparison of original(red), measured(blue), and filtered(green) signals.

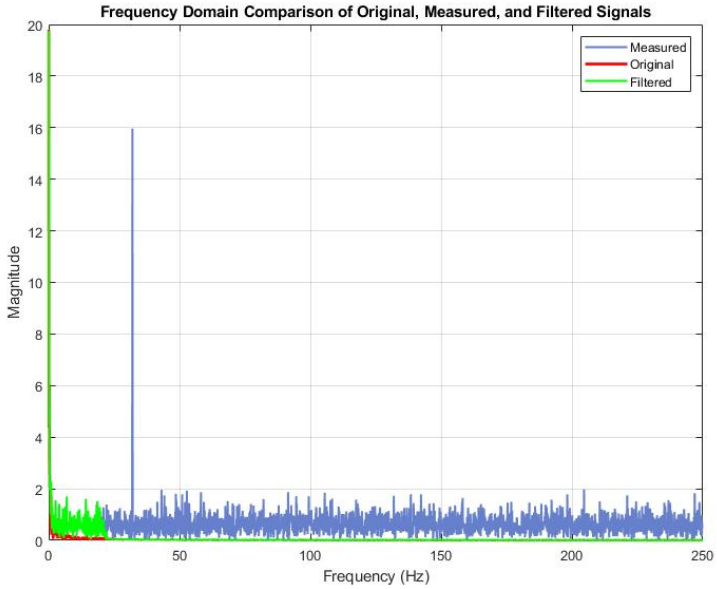

Figure 22: Frequency domain comparison of original(red), measured(blue), and filtered(green) signals.

The overall comparison confirms that the filtering process achieved its objectives. The low-pass filter effectively suppressed high-frequency noise while protecting the desired signal's low-frequency components. This performance could seen in both the time(Figure 21) and frequency(Figure 22) domain analyses.

3.2 Error Analysis

To quantify the performance of the filtering process, the Root Mean Square Error (RMSE) was calculated between the original and filtered signals. The RMSE provides a numerical measure of the average error introduced by the filtering process.RMSE was calculated as follows:

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_{\text{Orig},i} - y_{\text{Filt},i})^2} \tag{1}$$

We obtained that RMSE value is **13.79**, indicating a low average error and confirming the filter's effectiveness in preserving the original signal's characteristics. To analysing it visually, the time-varying error between the original and filtered signals, as shown in Figure 23.

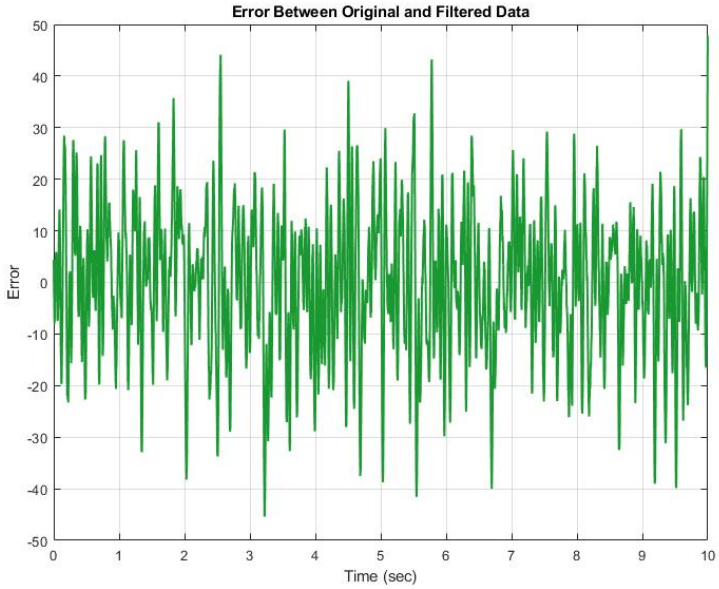

Figure 23: Time-varying error between the original and filtered signals.

The time-varying error plot highlights regions where the filtering process deviates slightly from the original signal, particularly at points of high frequency variation. Despite these small deviations, the overall RMSE value confirms that the filter successfully removed high-frequency noise.

3.3 Inverse Fourier Transform and Validation

After filtering, we analyzed the signal using the inverse Fourier transform (IFFT). This step was important to confirm the preservation of the original signal's characteristics in the time domain. By applying IFFT to the filtered signal, we converted it and compared it to the original signal. The results are demonstrated in Figures 24, 25, and 26.

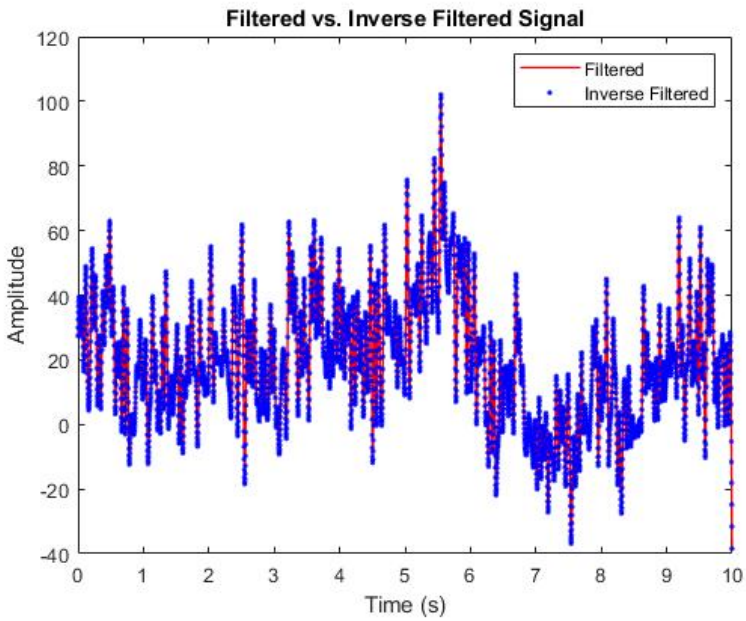

Figure 24: Filtered vs. inverse filtered signal. The comparison shows that the inverse filtered signal closely follows the filtered signal, indicating minimal loss during conversion.

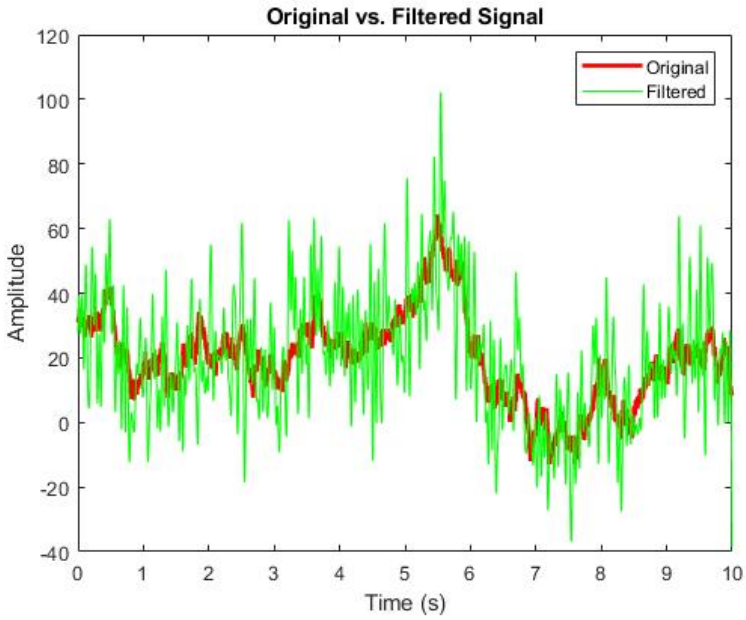

Figure 25: Original vs. filtered signal. The filtered signal effectively removes high-frequency noise while preserving the low-frequency characteristics of the original signal.

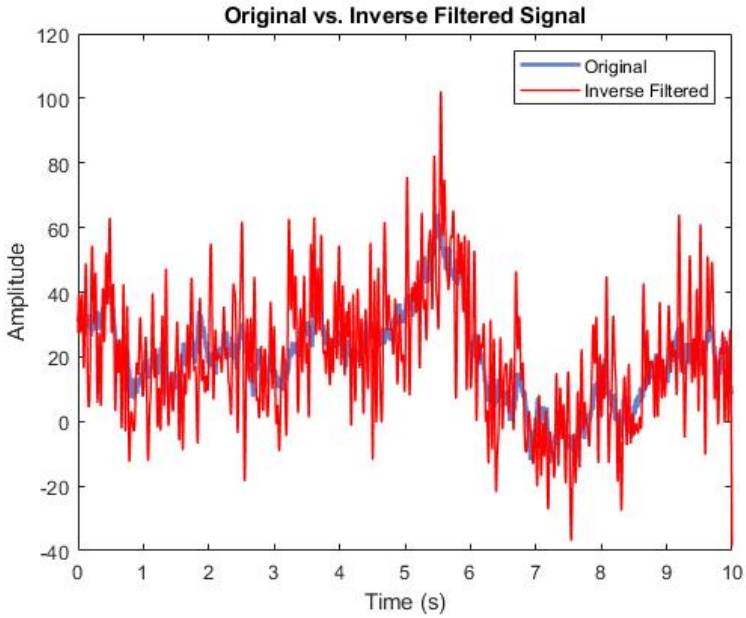

Figure 26: Original vs. inverse filtered signal. The inverse filtered signal closely resembles the original signal, confirming the accuracy of the filtering and conversion process.

Figure 24 shows the comparison between the filtered signal and the inverse filtered signal. The alignment between these signals indicates minimal conversion error during the IFFT process.

In Figure 25, the filtered signal (green) effectively eliminates noise components while retaining the original signal's (red) low-frequency features. This verifies the success of the filtering process in the time domain.

Finally, Figure 26 compares the original signal with the inverse filtered signal. The strong similarity between the two signals further validates that the IFFT step accurately converted the signal without introducing artifacts or distortions. This confirms that the filtering and conversion process successfully preserved the essential characteristics of the original data.

3.4 Noise Analysis and Potential Causes

The noise observed in the measured signal is likely caused by external interference or limitations of the measurement system. This noise predominantly resides in the higher frequency range, which justified the use of a low-pass filter with a cutoff frequency of 20 Hz. The frequency domain analysis, as depicted in Figure 22, illustrates the successful attenuation of these noise components, leaving the low-frequency signal largely unaffected.

3.5 Thinking about the Cause and Significance of Noise

The noise could be coming from any source. We could filter this circuit with an SG filter. The data we receive would then be biomedical data. We can think of the SG filter as a low pass filter. At the end of the day, all filters aim to optimise the noise in a data. For example, The SG filter has found broad-based acceptance across various applications, including but not limited to electroencephalography and ECG, elastography, near-infrared spectroscopy, functional magnetic resonance imaging, speech enhancement, and eye movement analysis [7]. As can be seen in these and many other examples, data filtering has an important place in our lives.

A MATLAB Code

The MATLAB code used to design and apply the low-pass filter is provided below:

```
% Define colors
red = [1, 0, 0];
green = [0, 1, 0];
blue = [0, 0, 1];
darkBlue = [0.1, 0.4, 0.7];
lightBlue = [0.4, 0.5, 0.8];

% Provided data: original, measured, and timevec
load GroupData_7.mat
% Simulation parameters
fs    = 1/mean(diff(timevec)); % Sampling frequency (from timevec)
npnts = length(timevec); % Number of points

% Our Given Signals
yOrig = original; % Original signal
y     = measurement; % Measured signal

% Power spectrum of the measured signal
yX = abs(fft(y)/npnts).^2; % Power spectrum
hz = linspace(0, fs/2, floor(npnts/2) + 1); % Frequency vector

% Plot the data
figure('Position', [300, 100, 800, 600]);

% Time domain plot
subplot(2, 1, 1);
plot(timevec, y, 'Color', lightBlue); % Measured signal
hold on;
plot(timevec, yOrig, 'LineWidth', 2, 'Color', red); % Original signal
xlabel('Time (sec)');
ylabel('Amplitude');
title('Time Domain');
legend({'Measured', 'Original'});
grid on;

% Frequency domain plot
subplot(2, 1, 2);
plot(hz, yX(1:length(hz)), 'k', 'LineWidth', 1); % Power spectrum
xlabel('Frequency (Hz)');
ylabel('Power');
title('Frequency Domain');
set(gca, 'YScale', 'log'); % Logarithmic scale for power
grid on;

% Lowpass filter
fcutoff = 20;
transw  = 0.2;
order   = round(10*fs/fcutoff);

shape   = [1 1 0 0];
frex    = [0 fcutoff fcutoff+fcutoff*transw fs/2] / (fs/2);

% Filter kernel
filtkern = firls(order, frex, shape);
```

```
% Filter kernel power spectrum
filtkernX = abs(fft(filtkern, npnts)).^2;

figure('Position', [300, 100, 800, 600]);

% Filter kernel plot
subplot(3, 2, 1); % Correct subplot syntax
plot((-order/2:order/2)/fs, filtkern, 'LineWidth', 3, 'Color', darkBlue);
xlabel('Time (s)');
title('Filter kernel');

% Filter kernel spectrum plot
subplot(3, 2, 2); hold on;
plot(frex*fs/2, shape, 'LineWidth', 1, 'Color', darkBlue);
plot(hz, filtkernX(1:length(hz)), 'LineWidth', 2, 'Color', darkBlue);
set(gca, 'XLim', [0 60]);
xlabel('Frequency (Hz)');
ylabel('Gain');
title('Filter kernel spectrum');
legend({'Ideal', 'Actual'});

% Applying filter to the data
yFilt = filtfilt(filtkern, 1, y);

% Power spectra of original and filtered signal
yOrigX = abs(fft(y)/npnts).^2;
yFiltX = abs(fft(yFilt)/npnts).^2;

subplot(3, 1, 2);
hold on; % Keep the original signal plotted
plot(timevec, y, 'Color', lightBlue, 'LineWidth', 1); % Measured (Original) Signal
plot(timevec, yFilt, 'LineWidth', 2, 'Color', red); % Filtered Signal
legend({'Measured Signal', 'Filtered Signal'});
xlabel('Time (sec.)');
ylabel('Amplitude');
grid on;

subplot(3, 1, 3);
plot(hz, yOrigX(1:length(hz)), 'Color', darkBlue);
hold on;
plot(hz, yFiltX(1:length(hz)), 'Color', green);
set(gca, 'xlim', [0 fs/5], 'yscale', 'log');
legend({'Signal', 'Filtered'});
xlabel('Frequency (Hz)');
ylabel('Power (log)');

% Apply inverse Fourier transform to the filtered data
invFilt = ifft(fft(yFilt));


figure;
plot(timevec, yFilt, 'Color', red, 'LineWidth', 1);
hold on;
plot(timevec, real(invFilt), '.', 'Color', blue, 'LineWidth', 1);
xlabel('Time (s)');
ylabel('Amplitude');
title('Filtered vs. Inverse Filtered Signal');
legend({'Filtered', 'Inverse Filtered'});
```

```
% Compare original and filtered signals
figure;
plot(timevec, yOrig, 'Color', red, 'LineWidth', 2);
hold on;
plot(timevec, yFilt, 'Color', green, 'LineWidth', 0.5);
xlabel('Time (s)');
ylabel('Amplitude');
title('Original vs. Filtered Signal');
legend({'Original', 'Filtered'});

% Compare original and inverse filtered signals
figure;
plot(timevec, yOrig, 'Color', lightBlue, 'LineWidth', 2);
hold on;
plot(timevec, real(invFilt), 'Color', red , 'LineWidth', 1);
xlabel('Time (s)');
ylabel('Amplitude');
title('Original vs. Inverse Filtered Signal');
legend({'Original', 'Inverse Filtered'});

% RMSE Calculation and Plot
% Compute RMSE
rmse = sqrt(mean((yOrig - yFilt).^2));

% Create a time-varying error signal
time_varying_error = yOrig - yFilt;

% Create a new figure for Time-varying Error
figure('Position', [300, 100, 800, 600]);
plot(timevec, time_varying_error, 'LineWidth', 1.5, 'Color', [0.1, 0.6, 0.2]);
xlabel('Time (sec)');
ylabel('Error');
title('Error Between Original and Filtered Data');
grid on;

% Comparison of Original, Measured, and Filtered Signals(Time Domain)
figure('Position', [300, 100, 800, 600]);
plot(timevec, y, 'Color', lightBlue, 'LineWidth', 1.5); % Measured signal
hold on;
plot(timevec, yOrig, 'Color', red, 'LineWidth', 2); % Original signal
plot(timevec, yFilt, 'Color', green, 'LineWidth', 1.5); % Filtered signal
xlabel('Time (sec)');
ylabel('Amplitude');
title('Time Domain Comparison of Original, Measured, and Filtered Signals');
legend({'Measured', 'Original', 'Filtered'});
grid on;

% Comparison of Original, Measured, and Filtered Signals(Frequency Domain)
figure('Position', [300, 100, 800, 600]);

% Compute the FFT of the signals
N = length(timevec);  % Number of samples
f = (0:N-1)*(fs/N);  % Frequency vector

% FFTs
Y = fft(y);           % FFT of measured signal
YOrig = fft(yOrig);   % FFT of original signal
```

```
YFilt = fft(yFilt);   % FFT of filtered signal

% Magnitude of the FFTs (normalize)
Y_mag = abs(Y) / N;
YOrig_mag = abs(YOrig) / N;
YFilt_mag = abs(YFilt) / N;

% Plot the magnitude of the FFTs up to Nyquist frequency
plot(f(1:floor(N/2)), Y_mag(1:floor(N/2)), 'Color', lightBlue, 'LineWidth', 1.5); % Measured signal
hold on;
plot(f(1:floor(N/2)), YOrig_mag(1:floor(N/2)), 'Color', red, 'LineWidth', 2); % Original signal
plot(f(1:floor(N/2)), YFilt_mag(1:floor(N/2)), 'Color', green, 'LineWidth', 1.5); % Filtered signal

xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Frequency Domain Comparison of Original, Measured, and Filtered Signals');
legend({'Measured', 'Original', 'Filtered'});
grid on;
```

References

[**1**] Proakis, J., and Manolakis, D. G., *Digital Signal Processing*, 4th ed., Prentice-Hall, 2007.

[**2**] Oppenheim, A. V., and Schafer, R. W., *Digital Signal Processing*, 2nd ed., Prentice-Hall, 1989.

[**3**] Jahan, I., and Sarker, O., *A Lower Transition Width FIR Filter and Its Noise Removal Performance on an ECG Signal*, American Journal of Engineering and Natural Sciences (AJENS), Vol. 1, No. 3, pp. 1, April 2017.

[**4**] Litwin, L., *FIR and IIR Digital Filters*, IEEE Potentials, vol. 19, no. 4, pp. 1-2, 2000.

[**5**] Jiamei Song, *Low-pass filter design and sampling theorem verification*, AIP Conference Proceedings, vol. 1971, no. 1, pp. 040017-4, 2018.

[**6**] MathWorks, "firls," MATLAB Documentation, 2024. Available: https://www.mathworks.com/help/signal/ref/firls. [Accessed: Dec. 17, 2024].

[**7**] Seddik, H., *A new family of Gaussian filters with adaptive lobe location and smoothing strength for efficient image restoration. EURASIP Journal on Advances in Signal Processing*, 2014.

8 Trimale, M., and Manish, B., *A Review: FIR Filter Implementation*, *Proc. 2nd IEEE Int. Conf. on Recent Trends in Electronics, Information & Communication Technology (RTEICT)*, IEEE, 2017.

9 Stanković, S., Stančić, P., Stančić, G., and Cvetković, S.,*Analysis of the Influence of Low-Pass Filter Magnitude and Phase Approximation Error on the Time-Domain Behavior,* in Proceedings of the 11th International Conference on Electrical, Electronics and Computer Engineering (IcETRAN), Niš, Serbia, June 3–6, 2024.

10 Shaikh, M. Z., Mal, K., Kumar, D., Ali, E., Chowdhry, B. S., and Baro, E. N.,*Simulation and Analysis of Ideal Low-Pass Filter Response Using MATLAB,* Mehran University Research Journal of Engineering and Technology, Vol. 42, No. 3, pp. 110–118, 2023, https://doi.org/10.22581/muet1982.2303.12.